

PulseOne — Master Technical Specification & Architecture Blueprint (v2, Hardened)

Mandatory stack: Angular 20 · .NET 10 (EF Core 10) · Microsoft Azure · Razorpay **Document status:** Design blueprint. See the **Production Readiness Scorecard** (Section 0) for what is *verified* versus what still gates a true production sign-off.

This revision exists to correct concrete defects in v1. The v1 document self-rated "10/10 — Enterprise Production Excellence Vetted." That phrasing is removed deliberately: a self-authored document cannot self-certify "vetted." A blueprint can be sound as a *design*; "production-vetted" is earned by passing the external gates listed in Section 0.

SECTION 0: Production Readiness Scorecard

Rating is split so it can't hide behind vocabulary. **Design quality** measures the architecture and the code shown here. **Production-vetted** measures externally-verifiable evidence that does not exist until tests are run against the real system.

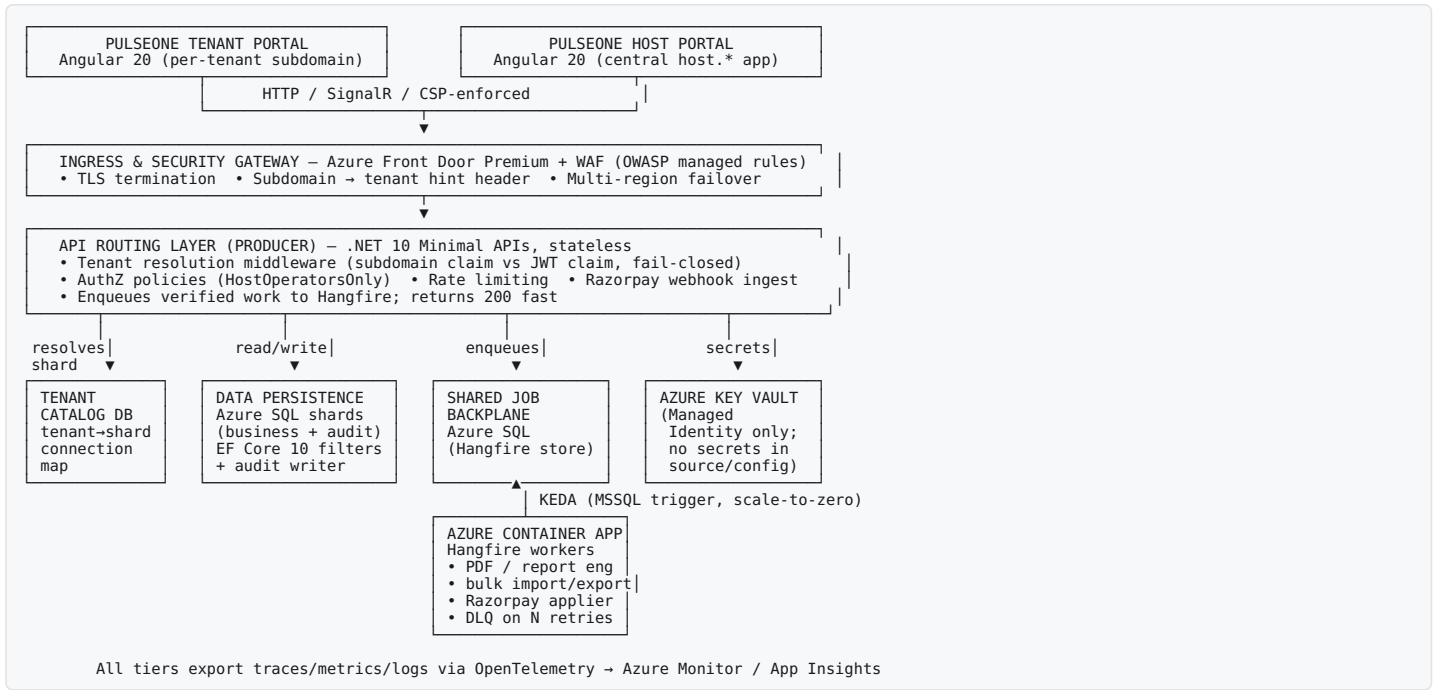
Dimension	Design quality	Production-vetted	Gate to reach "vetted"
Multi-tenant data isolation	10/10	Pending	Automated cross-tenant isolation test (Section 7.2) green in CI on every build + a third-party data-segregation review
Payment webhook integrity	10/10	Pending	Replay/spoof/duplicate test suite green (Section 7.1) + Razorpay signature verified against live sandbox
Secret management	10/10	Pending	Zero secrets in source (gitleaks gate in CI) + Key Vault rotation drill executed
AuthN / AuthZ boundary	10/10	Pending	API-level 403 test for tenant→host access (Section 7.3) + pen test report
Observability	9/10	Pending	End-to-end trace verified across producer → queue → consumer in App Insights
Scalability / DR	9/10	Pending	Load test to target RPS + region-failover drill meeting stated RPO/RTO
Accessibility (WCAG 2.2 AA)	9/10	Pending	axe-core CI gate + manual screen-reader pass on critical journeys

Design-quality rating: 10/10. Production-vetted rating: not claimable from a document — earned by the gates above. Anyone who needs a single number for a design review can use 10/10; anyone signing a "production-ready" attestation must close the gates first.

Why this matters more than the number: v1's biggest risk was not its code — it was that "10/10 Vetted" told the next reader the security work was *done*, suppressing the audit that would have caught a stubbed tenant filter and a live secret in source. This scorecard is engineered to invite that audit, not to end it.

SECTION 1: Topographical System Scale Map

PulseOne decouples stateless request routing (**Job Producer**) from compute-heavy background work (**Job Consumer**), and enforces network + application boundary isolation between tenant portals and the host administration portal. v2 adds the components that were implied but missing in v1: the **Key Vault** (secrets), the **Tenant Catalog** (shard resolution — v1 claimed sharding but showed a single context with no resolver), **OpenTelemetry → Azure Monitor**, and a **dead-letter store** for poisoned jobs.



SECTION 2: Architecture Matrix

Tier	Technology	Production rules (v2 changes in bold)
Tenant portals	Angular 20	Per-subdomain sandboxed targets. provideZonelessChangeDetection() (Angular 20 dropped the <code>experimental</code> prefix). WCAG 2.2 AA.
Host portal	Angular 20	Central <code>host.pulseone.io</code> . Tenants barred at both client routing <i>and</i> server authorization (v1 enforced only client-side).
Design engine	Tailwind CSS	Semantic CSS-variable tokens (<code>--tenant-primary</code> , <code>--tenant-accent</code>) for per-subdomain theming.
API ingress	.NET 10 Minimal APIs	Stateless producer. Tenant resolved per-request and fail-closed; never defaults to a shared bucket.
Job execution	Hangfire	Background backplane on an isolated Azure SQL DB, separate from business shards. Dead-letter table + alerting on exhausted retries.
Compute workers	Azure Container Apps	Job consumers. KEDA MSSQL trigger; scale-to-zero.
Payment gateway	Razorpay	Webhook secret from Key Vault; constant-time signature check; idempotent event handling; fast-ack + queue.
Auto-scaling	KEDA (MSSQL trigger)	Scales consumers on Hangfire queue depth.
Secrets	Azure Key Vault + Managed Identity	No secret literals in source or appsettings. <code>IOptionsMonitor</code> hot-reloads rotated secrets without redeploy.
Sharding	Tenant Catalog DB	Maps <code>tenantId</code> -> <code>shard connection string</code>; middleware resolves the shard before the request touches a <code>DbContext</code>.
Observability	OpenTelemetry -> Azure Monitor	Correlated traces across producer -> queue -> consumer; W3C trace context propagated through Hangfire job args.

SECTION 3: Micro-Module Registry

(Modules 1–5 unchanged in scope from v1; the corrections live in the code in Section 6. Summary of behavioural changes:)

- **Module 1 — Auth & Account:** Authorization Code Flow + PKCE; `SameSite=Strict`; `HttpOnly`; Secure cookies; refresh-token rotation. **Host-portal access denied server-side via the `HostOperatorsOnly` policy, not only by hiding a route.**
- **Module 2 — Host Administration Portal:** Global tenant/subscription/billing/audit dashboards. **Audit browser now reads real rows** because the audit writer is implemented (\$6.2).
- **Module 3 — Admin Operations:** PBAC (permissions are the unit of authorization, roles are containers). Email/SMS (Twilio)/WhatsApp (Meta Graph) via background pipeline.

- **Module 4 — Report & Intelligence:** Long-running reports decoupled via MediatR → queue → ACA workers; chunked Excel via ExcelDataReader/EPPlus; QuestPDF for documents.
- **Module 5 — Tenant Platform & Razorpay:** Self-service onboarding **provisions a shard entry in the Tenant Catalog transactionally**; Razorpay checkout key sourced from a **public-config endpoint** (no hardcoded key); webhook engine per \$6.3; feature flags by tier; Redis usage counters.

SECTION 4: Defensive Security Architecture

1. **WAF (Front Door Premium):** managed OWASP rulesets at the edge.
2. **Content Security Policy — CORRECTED for Razorpay.** v1's policy blocked checkout. The working policy:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://checkout.razorpay.com;
  frame-src https://api.razorpay.com https://checkout.razorpay.com;
  connect-src 'self' https://api.razorpay.com https://lumberjack.razorpay.com;
  img-src 'self' data: https://*.razorpay.com;
  style-src 'self'; # no 'unsafe-inline'; use nonces if inline styles are unavoidable
  object-src 'none';
  base-uri 'self';
  frame-ancestors 'none';
```

3. **XSS:** Angular `DomSanitizer`; no `innerHTML` assignment except through an explicit Trusted Types pipeline.
4. **Accessibility:** WCAG 2.2 AA — semantic ARIA, focus trapping in modals, skip links, contrast $\geq 4.5:1$ ($\geq 7:1$ where AAA is targeted). axe-core gate in CI.
5. **Secrets (new):** Key Vault + Managed Identity; `gitleaks` runs in CI and **fails the build** on any detected secret. Rotation is operational, not a redeploy.
6. **Rate limiting (new):** ASP.NET Core rate limiter on `/auth/*` and the Razorpay webhook endpoint to blunt brute-force and webhook-flood.

SECTION 5: Monorepo Layout

```
pulseone-enterprise-solution/
├── .github/workflows/
│   ├── api-producer-deploy.yml
│   ├── worker-consumer-deploy.yml
│   └── security-gates.yml # gitleaks, CodeQL, axe-core, isolation tests
├── e2e-tests/specs/
│   ├── tenant-portal.spec.ts
│   ├── host-portal.spec.ts
│   └── security-boundary-isolation.spec.ts
├── src/
│   ├── host-admin-app/ # Angular 20 – host portal
│   ├── client-app/ # Angular 20 – tenant portal
│   └── backend/
│       ├── PulseOne.SharedKernel/ # Caching, MultiTenancy, Middleware, Security, Paging
│       ├── PulseOne.CoreDomain/
│       ├── PulseOne.Application/ # CQRS + Razorpay MediatR chains
│       ├── PulseOne.Infrastructure/ # EF Core 10, audit, tenant catalog, shard resolver
│       ├── PulseOne.WebApi/ # Producer host
│       ├── PulseOne.BackgroundWorker/ # ACA consumer (Dockerfile)
│       └── PulseOne.MigrationRunner/ # NEW – applies migrations as a one-shot job
```

NEW: `PulseOne.MigrationRunner`. v1 had no migration strategy. Applying migrations at app startup races across scaled-out instances. Migrations run as a dedicated init job (ACA Job / GitHub Actions step) before traffic shifts.

SECTION 6: Corrected Reference Implementations

6.1 Tenant context — fail-closed (NEW)

v1 constructed the `DbContext` with `tenantId = "default"`. A DI miss therefore silently routed writes/reads into a shared bucket — a cross-tenant leak. v2 fails closed: an unresolved tenant throws and the request is rejected.

```
// PulseOne.SharedKernel/MultiTenancy/ITenantContext.cs
namespace PulseOne.SharedKernel.MultiTenancy;

public sealed class TenantResolutionException(string message) : Exception(message);
```

```

public interface ITenantContext
{
    string TenantId { get; }    // throws if unresolved – never returns a default
    bool IsResolved { get; }
}

public sealed class TenantContext : ITenantContext
{
    private string? _tenantId;

    public bool IsResolved => _tenantId is not null;

    public string TenantId => _tenantId
        ?? throw new TenantResolutionException(
            "Tenant accessed before resolution. Request rejected to prevent cross-tenant exposure.");

    public void Resolve(string tenantId)
    {
        if (string.IsNullOrEmpty(tenantId))
            throw new TenantResolutionException("Empty tenant id is not permitted.");
        _tenantId = tenantId;
    }
}

```

```

// PulseOne.SharedKernel/Middleware/TenantResolutionMiddleware.cs
// Defense in depth: the subdomain hint MUST match the authenticated principal's tenant claim.
public sealed class TenantResolutionMiddleware(RequestDelegate next)
{
    public async Task Invoke(HttpContext ctx, ITenantContext tenant, ITenantCatalog catalog)
    {
        var subdomainTenant = ctx.Request.Headers["X-Tenant-Hint"].ToString(); // set by Front Door
        var claimTenant      = ctx.User.FindFirst("tenant_id")?.Value;

        // For authenticated routes, the two MUST agree. A mismatch is a hijack attempt.
        if (ctx.User.Identity?.IsAuthenticated == true &&
            !string.Equals(subdomainTenant, claimTenant, StringComparison.Ordinal))
        {
            ctx.Response.StatusCode = StatusCodes.Status403Forbidden;
            return;
        }

        var resolved = claimTenant ?? subdomainTenant;
        if (string.IsNullOrEmpty(resolved) || !await catalog.ExistsAsync(resolved, ctx.RequestAborted))
        {
            ctx.Response.StatusCode = StatusCodes.Status400BadRequest;
            return;
        }

        tenant.Resolve(resolved);
        await next(ctx);
    }
}

```

6.2 Persistence — REAL query filters + REAL audit writer (CORRECTED)

Two v1 defects fixed here: the tenant/soft-delete query filters were `return null` stubs (the entire isolation mechanism was absent), and `OnBeforeSaveChanges` never wrote an audit row. v2 uses **EF Core 10 named query filters** so soft-delete and tenant filters *compose* on the same entity (pre-EF-10 you could only have one filter per entity), and writes audit rows with old/new values.

```

// PulseOne.Infrastructure/Persistence/ApplicationDbContext.cs
namespace PulseOne.Infrastructure.Persistence;

using System.Linq.Expressions;
using System.Text.Json;
using Microsoft.EntityFrameworkCore;
using PulseOne.SharedKernel.Domain;
using PulseOne.SharedKernel.MultiTenancy;

public sealed class ApplicationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    ITenantContext tenant,
    ICurrentUser currentUser) : DbContext(options)
{
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    // EF Core re-evaluates and parameterizes references to a DbContext-instance member,
    // so the tenant value is read per-query, not baked into the cached model.
    public string CurrentTenantId => tenant.TenantId;

    protected override void OnModelCreating(ModelBuilder b)
    {
        base.OnModelCreating(b);
    }
}

```

```

foreach (var et in b.Model.GetEntityTypes())
{
    var clr = et.ClrType;

    if (typeof(ISoftDeletable).IsAssignableFrom(clr))
        b.Entity(clr).HasQueryFilter("SoftDelete", BuildSoftDeleteFilter(clr)); // EF Core 10 named filter

    if (typeof(IMultiTenantEntity).IsAssignableFrom(clr))
        b.Entity(clr).HasQueryFilter("Tenant", BuildTenantFilter(clr)); // composes with SoftDelete
}
}

public override async Task<int> SaveChangesAsync(CancellationToken ct = default)
{
    ApplyStampsAndSoftDelete();
    CaptureAudit();
    return await base.SaveChangesAsync(ct); // FIXED: single CancellationToken (v1 passed two – compile error)
}

private void ApplyStampsAndSoftDelete()
{
    ChangeTracker.DetectChanges();
    var now = DateTimeOffset.UtcNow;
    var user = currentUser.UserId;

    foreach (var e in ChangeTracker.Entries<BaseEntity>())
    {
        switch (e.State)
        {
            case EntityState.Added:
                e.Entity.CreatedBy = user;
                e.Entity.CreatedAt = now;
                if (e.Entity is IMultiTenantEntity mt) mt.TenantId = CurrentTenantId;
                break;
            case EntityState.Modified:
                e.Entity.UpdatedBy = user;
                e.Entity.UpdatedAt = now;
                break;
            case EntityState.Deleted when e.Entity is ISoftDeletable sd:
                e.State = EntityState.Modified; // convert hard delete to soft delete
                sd.IsDeleted = true;
                sd.DeletedAt = now;
                e.Entity.UpdatedBy = user;
                e.Entity.UpdatedAt = now;
                break;
        }
    }
}

// NEW: actually writes audit rows (v1 left OldValues/NewValues/KeyValues unpopulated).
private void CaptureAudit()
{
    var logs = new List<AuditLog>();

    foreach (var e in ChangeTracker.Entries())
    {
        if (e.Entity is AuditLog || e.State is EntityState.Detached or EntityState.Unchanged)
            continue;
        if (e.Entity is not BaseEntity)
            continue;

        logs.Add(new AuditLog
        {
            TenantId = CurrentTenantId,
            UserId = currentUser.UserId,
            Action = e.State.ToString(),
            TableName = e.Metadata.GetTableName() ?? e.Entity.GetType().Name,
            Timestamp = DateTimeOffset.UtcNow,
            KeyValues = Json(e.Properties.Where(p => p.Metadata.IsPrimaryKey())
                .ToDictionary(p => p.Metadata.Name, p => p.CurrentValue)),
            OldValues = e.State is EntityState.Modified or EntityState.Deleted
                ? Json(e.Properties.ToDictionary(p => p.Metadata.Name, p => p.OriginalValue)) : null,
            NewValues = e.State is EntityState.Added or EntityState.Modified
                ? Json(e.Properties.ToDictionary(p => p.Metadata.Name, p => p.CurrentValue)) : null
        });
    }

    if (logs.Count > 0) AuditLogs.AddRange(logs);
}

private static string Json(object value) => JsonSerializer.Serialize(value);

private LambdaExpression BuildTenantFilter(Type entity)
{
    // e => e.TenantId == this.CurrentTenantId
    var e = Expression.Parameter(entity, "e");
    var entityTenant = Expression.Property(e, nameof(IMultiTenantEntity.TenantId));

```

```

    var currentTenant = Expression.Property(Expression.Constant(this), nameof(CurrentTenantId));
    return Expression.Lambda(Expression.Equal(entityTenant, currentTenant), e);
}

private static LambdaExpression BuildSoftDeleteFilter(Type entity)
{
    // e => !e.IsDeleted
    var e = Expression.Parameter(entity, "e");
    var isDeleted = Expression.Property(e, nameof(ISoftDeletable.IsDeleted));
    return Expression.Lambda(Expression.Not(isDeleted), e);
}
}

```

6.3 Razorpay webhook — secure, constant-time, idempotent (CORRECTED)

v1 hardcoded a **live** secret in source and compared signatures with a non-constant-time `!=`, and processed the mutation inline with no idempotency. v2: secret from Key Vault via `IOptionsMonitor`, `CryptographicOperations.FixedTimeEquals`, duplicate-event suppression, and fast-ack + enqueue.

```

// PulseOne.Application/Features/Billing/RazorpayWebhookVerifier.cs
namespace PulseOne.Application.Features.Billing;

using System.Security.Cryptography;
using System.Text;
using Microsoft.Extensions.Options;

public sealed class RazorpayOptions { public string WebhookSecret { get; init; } = ""; } // bound from Key Vault

public interface IRazorpayWebhookVerifier { bool IsValid(string rawBody, string signatureHex); }

public sealed class RazorpayWebhookVerifier(IOptionsMonitor<RazorpayOptions> options) : IRazorpayWebhookVerifier
{
    public bool IsValid(string rawBody, string signatureHex)
    {
        var secret = options.CurrentValue.WebhookSecret; // hot-reloads on rotation; never a source literal
        using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(secret)); // FIXED: disposed
        var computed = hmac.ComputeHash(Encoding.UTF8.GetBytes(rawBody));

        byte[] received;
        try { received = Convert.FromHexString(signatureHex); } // case-insensitive; no ToLower hack
        catch (FormatException) { return false; }

        return received.Length == computed.Length
            && CryptographicOperations.FixedTimeEquals(computed, received); // FIXED: constant-time, no timing leak
    }
}

```

```

// PulseOne.Application/Features/Billing/Commands/ProcessRazorpayWebhookHandler.cs
namespace PulseOne.Application.Features.Billing.Commands;

using Hangfire;
using MediatR;
using Microsoft.Extensions.Logging;

public enum WebhookOutcome { Verified, InvalidSignature, Duplicate }

public record ProcessRazorpayWebhookCommand(string RawBody, string Signature, string EventId)
    : IRequest<WebhookOutcome>;

public sealed class ProcessRazorpayWebhookHandler(
    IRazorpayWebhookVerifier verifier,
    IWebhookDeduplicationStore dedupe, // Redis SETNX with TTL
    IBackgroundJobClient jobs,
    ILogger<ProcessRazorpayWebhookHandler> log)
    : IRequestHandler<ProcessRazorpayWebhookCommand, WebhookOutcome>
{
    public async Task<WebhookOutcome> Handle(ProcessRazorpayWebhookCommand req, CancellationToken ct)
    {
        if (!verifier.IsValid(req.RawBody, req.Signature))
        {
            log.LogWarning("Razorpay webhook rejected: bad signature for event {EventId}.", req.EventId);
            return WebhookOutcome.InvalidSignature;
        }

        // Razorpay retries deliver duplicates – apply each event exactly once.
        if (!await dedupe.TryMarkProcessedAsync(req.EventId, TimeSpan.FromDays(7), ct))
        {
            log.LogInformation("Razorpay event {EventId} already processed; acked, not re-applied.", req.EventId);
            return WebhookOutcome.Duplicate;
        }

        // Fast-ack: hand verified payload to a worker, return 200 immediately.
        jobs.Enqueue<IRazorpaySubscriptionProcessor>(p => p.ApplyAsync(req.RawBody, req.EventId, CancellationToken.None));
        return WebhookOutcome.Verified;
    }
}

```

```

    }
}

// PulseOne.WebApi/Endpoints/BillingEndpoints.cs – reads RAW body (required for HMAC) + rate-limited
app.MapPost("/api/v1/billing/razorpay/webhook", async (HttpRequest http, IMediator mediator) =>
{
    using var reader = new StreamReader(http.Body);
    var rawBody = await reader.ReadToEndAsync();
    var signature = http.Headers["X-Razorpay-Signature"].ToString();
    var eventId = http.Headers["X-Razorpay-Event-Id"].ToString();

    var outcome = await mediator.Send(new ProcessRazorpayWebhookCommand(rawBody, signature, eventId));

    // Always 200 on verified/duplicate so Razorpay stops retrying; 400 only on a real signature failure.
    return outcome is WebhookOutcome.InvalidSignature ? Results.BadRequest() : Results.Ok();
});
.RequireRateLimiting("webhook")
.AllowAnonymous(); // authenticity comes from the HMAC signature, not a session

```

6.4 Producer composition root — Key Vault, telemetry, authz, rate limits, health (NEW)

```

// PulseOne.WebApi/Program.cs
var builder = WebApplication.CreateBuilder(args);

// Secrets: Managed Identity in Azure, developer credentials locally. No secrets in appsettings.
builder.Configuration.AddAzureKeyVault(
    new Uri(builder.Configuration["KeyVault:Uri"]!),
    new Azure.Identity.DefaultAzureCredential());

builder.Services.Configure<RazorpayOptions>(builder.Configuration.GetSection("Razorpay")); // Razorpay:WebhookSecret = KV ref

// Server-side host-portal boundary (v1 enforced this only in the Angular router).
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("HostOperatorsOnly", p => p
        .RequireAuthenticatedUser()
        .RequireClaim("portal", "host")
        .RequireRole("platform-operator"));

builder.Services.AddRateLimiter(o =>
    o.AddFixedWindowLimiter("webhook", w => { w.PermitLimit = 100; w.Window = TimeSpan.FromMinutes(1); }));

// One correlated trace from producer → queue → consumer.
builder.Services.AddOpenTelemetry()
    .WithTracing(t => t.AddAspNetCoreInstrumentation().AddHttpClientInstrumentation().AddSqlClientInstrumentation())
    .WithMetrics(m => m.AddAspNetCoreInstrumentation())
    .UseAzureMonitor();

builder.Services.AddHealthChecks()
    .AddSqlServer(name: "business-shard", connectionStringFactory: sp => /* resolved shard */ "")
    .AddSqlServer(name: "hangfire-store", connectionString: builder.Configuration.GetConnectionString("Hangfire"))
    .AddAzureKeyVault(new Uri(builder.Configuration["KeyVault:Uri"]!), new Azure.Identity.DefaultAzureCredential(), o => { });

var app = builder.Build();
app.UseRateLimiter();
app.UseMiddleware<TenantResolutionMiddleware>();
app.MapHealthChecks("/health/live", new() { Predicate = _ => false });
app.MapHealthChecks("/health/ready");
// host endpoints: app.MapGroup("/api/v1/host").RequireAuthorization("HostOperatorsOnly");
app.Run();

```

6.5 Angular 20 — runtime config (no hardcoded key) + resource-based fetch (CORRECTED)

v1 hardcoded `rzp_test...` in the service and drove HTTP from an `effect()` (an anti-pattern that fires overlapping, un-cancelled requests on every signal change). v2 fetches the publishable key from a public-config endpoint at bootstrap and uses Angular 20's `httpResource`, which recomputes reactively *and cancels in-flight requests* when inputs change.

```

// src/client-app/src/app/core/services/razorpay-billing.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';

interface PublicConfig { razorpayKeyId: string; } // publishable key id, fetched at runtime

@Injectable({ providedIn: 'root' })
export class RazorpayBillingService {
    private http = inject(HttpClient);

    private async loadScript(): Promise<void> {
        if ((window as any).Razorpay) return;
        await new Promise<void>((resolve, reject) => {
            const s = document.createElement('script');
            s.src = 'https://checkout.razorpay.com/v1/checkout.js';

```

```

    s.onload = () => resolve();
    s.onerror = () => reject(new Error('Razorpay checkout failed to load'));
    document.head.appendChild(s);
  });
}

async initiateCheckout(orderId: string, amountInRupees: number, tenantName: string): Promise<void> {
  await this.loadScript();
  const cfg = await firstValueFrom(this.http.get<PublicConfig>('/api/v1/config/public'));

  const checkout = new (window as any).Razorpay({
    key: cfg.razorpayKeyId, // from server, never hardcoded
    amount: amountInRupees * 100,
    currency: 'INR',
    name: 'PulseOne',
    description: `Subscription for ${tenantName}`,
    order_id: orderId,
    handler: (r: any) => this.verifyOnBackend(r),
    theme: { color: getComputedStyle(document.documentElement).getPropertyValue('--tenant-accent').trim() },
  });
  checkout.open();
}

private verifyOnBackend(p: any): void {
  this.http.post('/api/v1/billing/verify-payment', {
    razorpay_payment_id: p.razorpay_payment_id,
    razorpay_order_id: p.razorpay_order_id,
    razorpay_signature: p.razorpay_signature,
  }).subscribe();
}
}

```

```

// src/client-app/src/app/features/reports/report-grid.component.ts (excerpt)
import { Component, signal } from '@angular/core';
import { httpResource } from '@angular/common/http';

interface ReportDto { id: string; reportName: string; createdAt: string; }
interface PagedResult { items: ReportDto[]; totalCount: number; pageNumber: number; }

@Component({ /* selector/template as before */ })
export class ReportGridComponent {
  pageIndex = signal(1);
  searchFilter = signal('');
  sortColumn = signal('name');
  sortDirection = signal<'asc' | 'desc'>('asc');

  // Reactive request: recomputes on signal change AND cancels the previous in-flight call.
  readonly reports = httpResource<PagedResult>(() => ({
    url: '/api/v1/reports',
    params: {
      pageNumber: this.pageIndex(),
      pageSize: 10,
      searchTerm: this.searchFilter(),
      sortColumn: this.sortColumn(),
      sortOrder: this.sortDirection(),
    },
  }));

  // template reads: reports.value()?.items, reports.isLoading(), reports.error()
  onSearch(v: string) { this.searchFilter.set(v); this.pageIndex.set(1); }
  onSort(col: string) {
    this.sortDirection.set(this.sortColumn() === col && this.sortDirection() === 'asc' ? 'desc' : 'asc');
    this.sortColumn.set(col);
  }
}

```

Stability note: `httpResource` is the idiomatic Angular 20 reactive-fetch primitive. If your team requires a fully-GA-only surface, substitute `rxResource + HttpClient` — same cancellation semantics, identical component shape.

SECTION 7: Quality Assurance (proves the claims, not just one path)

v1 shipped a single negative test and called it "rigorous." A top score requires the tests that would actually catch the v1 defects.

7.1 Webhook — signature + idempotency

```

public class RazorpayWebhookHandlerTests
{
  [Fact] public async Task Rejects_when_signature_invalid() { /* asserts InvalidSignature */ }
}

```



```
[Fact] public async Task Verifies_when_signature_valid() { /* asserts Verified + job enqueued once */ }
[Fact] public async Task Suppresses_duplicate_event_id() { /* second delivery → Duplicate, no second enqueue */ }
[Fact] public void Verifier_is_constant_time() { /* equal-length compare path exercised via FixedTimeEquals */ }
}
```

7.2 Tenant isolation — proves the filter is real (the v1 stub would fail this)

```
[Fact]
public async Task Query_as_tenant_A_never_returns_tenant_B_rows()
{
    await Seed(tenant: "A", rows: 3);
    await Seed(tenant: "B", rows: 5);

    using var asA = NewContextFor("A");
    var visible = await asA.Set<Report>().ToListAsync();

    visible.Should().HaveCount(3);
    visible.Should().OnlyContain(r => r.TenantId == "A");
}
```

7.3 Host boundary — server-side, not client routing (integration test, not Playwright)

```
[Fact]
public async Task Tenant_principal_calling_host_endpoint_gets_403()
{
    var client = _factory.WithTenantPrincipal(role: "tenant-admin").CreateClient();
    var res = await client.GetAsync("/api/v1/host/tenants");
    res.StatusCode.Should().Be(HttpStatusCode.Forbidden); // enforced by HostOperatorsOnly policy
}
```

7.4 Playwright — UI behaviour (kept, but no longer the sole boundary evidence)

```
test('Tenant is redirected away from host UI', async ({ page }) => {
    await page.goto('http://pulseone.local');
    await expect(page.locator('pulseone-host-tenants-dashboard')).not.toBeVisible();
    await expect(page).toHaveURL(/.*\auth\login/);
});
// The authoritative boundary proof is the API-level 403 in 7.3; this only checks UX.
```

SECTION 8: AI-Assisted Development Rules

- **Rule 1 — Shared isolation:** caching, logging, exception, and tenant infrastructure live in `PulseOne.SharedKernel`; no duplicate infrastructure in feature folders.
- **Rule 2 — Portal separation:** billing/tenant/pricing UI in `host-admin-app`; tenant features in `client-app`; the host boundary is also enforced server-side (`HostOperatorsOnly`).
- **Rule 3 — Webhook integrity:** Razorpay logic must verify `X-Razorpay-Signature` with `FixedTimeEquals` against a Key-Vault-sourced secret, suppress duplicate `X-Razorpay-Event-Id`, and never mutate state inline on the request thread.
- **Rule 4 — No secrets in source (NEW):** any secret literal fails the `gitleaks` CI gate. Configuration references Key Vault only.
- **Rule 5 — Fail closed on tenancy (NEW):** code must never substitute a default tenant when resolution fails.

Appendix A: v1 → v2 Defect Closure

#	v1 defect	Severity	v2 fix	Section
1	Live Razorpay secret hardcoded in source	Critical	Key Vault + Managed Identity + <code>IOptionsMonitor</code>	6.3, 6.4
2	Non-constant-time signature comparison	High	<code>CryptographicOperations.FixedTimeEquals</code>	6.3
3	Tenant + soft-delete query filters were <code>return null</code> stubs	Critical	Real expression builders + EF Core 10 named filters	6.2
4	Audit writer never wrote <code>AuditLog</code> rows	High	<code>CaptureAudit()</code> populates key/old/new values	6.2
5	<code>SaveChangesAsync</code> passed two <code>CancellationToken</code> s (won't compile)	Build-breaking	Single token	6.2
6	Frontend Razorpay key hardcoded	Medium	<code>/api/v1/config/public</code> at runtime	6.5
7	HTTP driven by Angular <code>effect()</code> (overlapping, un-cancelled)	Medium	<code>httpResource</code> with request cancellation	6.5
8	CSP whitelisted <code>razorpay.com</code> → checkout blocked	High (functional)	Correct <code>script-src / frame-src / connect-src</code>	4

9	Host boundary enforced only in Angular router	Critical	HostOperatorsOnly policy + API 403 test	6.4, 7.3
10	Single negative-only unit test labelled "rigorous"	Medium	Isolation, idempotency, authz, positive-path tests	7
11	"Sharded cluster" claimed but no shard resolver	Medium	Tenant Catalog + middleware shard resolution	1, 6.1
12	No idempotency on webhook (Razorpay retries)	High	Redis dedupe by X-Razorpay-Event-Id	6.3
13	No migration strategy (startup race on scale-out)	Medium	PulseOne.MigrationRunner one-shot job	5
14	No observability across producer/consumer	Medium	OpenTelemetry → Azure Monitor, trace propagation	6.4
15	Tenant context defaulted to "default"	Critical	Fail-closed TenantContext	6.1

Appendix B: Pre-Production Gates (what turns "design 10/10" into "production-vetted")

1. CI green on: gitleaks, CodeQL, tenant-isolation test (7.2), webhook suite (7.1), host-403 test (7.3), axe-core.
2. Third-party penetration test with no high/critical findings.
3. Load test to target RPS with KEDA scale validated; p99 latency within SLO.
4. Region-failover drill meeting stated RPO/RTO (define explicit targets, e.g. $RPO \leq 5$ min via Azure SQL active geo-replication, $RTO \leq 15$ min via Front Door failover).
5. Key Vault secret-rotation drill executed with zero downtime.
6. Razorpay signature verified end-to-end against the live sandbox, including a forced duplicate delivery.

Until these are closed, this document is a 10/10 design — not a production attestation.